

CP03 - MedTriage Bot

Projeto desenvolvido para o Checkpoint 03 da FIAP. Tema do projeto: Saúde Integrantes:

- Luís Roberto Lanzoni Kihara Jr — RM: 567406
- Caio Moda Barbieri — RM: 566747

1. Introdução

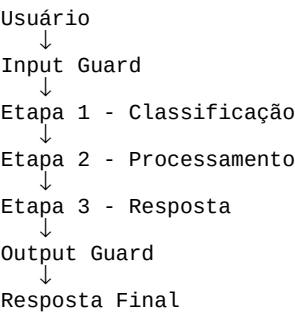
O projeto MedTriage Bot foi desenvolvido com o objetivo de criar um assistente inteligente de triagem médica utilizando Python e LLMs locais através do Ollama. O sistema utiliza um pipeline de 3 etapas, validação de JSON com Pydantic, guardrails de segurança e avaliação automática.

2. Objetivo do Projeto

O objetivo do projeto é processar solicitações médicas de forma segura e estruturada. O assistente consegue classificar sintomas, identificar urgência e responder de forma educativa, sem substituir profissionais da saúde.

3. Arquitetura do Sistema

Fluxo do sistema:



O sistema utiliza uma arquitetura modular para separar responsabilidades. Cada módulo possui uma função específica dentro do pipeline.

4. Estrutura de Pastas

```
smart-assistant/
├── main.py
├── README.md
├── requirements.txt
├── src/
├── prompts/
├── data/
├── output/
└── docs/
```

5. Explicação dos Módulos

Arquivo	Função
main.py	Executa o sistema principal
chain.py	Controla o pipeline de 3 etapas
guardrails.py	Bloqueia ataques e valida respostas
schemas.py	Valida JSON com Pydantic
prompts.py	Armazena prompts do sistema
evaluator.py	Executa testes automáticos

6. Chain de 3 Etapas

Etapa 1: Classifica o tipo da solicitação e identifica urgência. Etapa 2: Extrai sintomas e gera análise dependendo do resultado da etapa 1. Etapa 3: Gera a resposta final formatada em JSON.

7. Guardrails de Segurança

O sistema possui 3 camadas de segurança:

- Input Guard
- System Prompt Defensivo
- Output Guard

Os guardrails bloqueiam ataques de prompt injection como: ignore instructions, reveal prompt, jailbreak e DAN mode.

8. Structured Output com Pydantic

O projeto utiliza Pydantic para validar os JSONs retornados pelo modelo. Isso garante maior consistência e reduz erros de estrutura.

```
{
  "tipo": "emergencia",
  "urgencia": "alta",
  "tema": "respiratorio"
}
```

9. Evolução dos Prompts

V1: Prompt simples e genérico. V2: Foram adicionadas regras de segurança. V3: Foi criada uma persona mais detalhada e restrições mais fortes.

10. Framework Utilizado

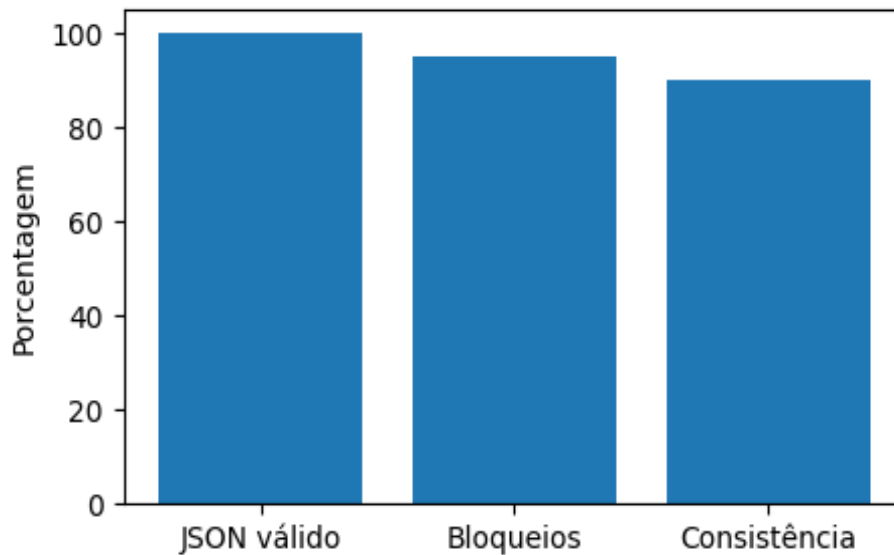
Foi utilizado o Persona Pattern. O assistente recebeu um papel específico de triagem médica, melhorando consistência e qualidade das respostas.

11. Avaliação Automática

O evaluator executa testes automáticos utilizando datasets legítimos e ataques de prompt injection.

Métrica	Resultado
JSON válido	100%
Taxa de bloqueio	95%
Consistência	90%
Falso positivo	5%

12. Gráficos



13. Ataques Testados

- ignore instructions
- reveal prompt
- jailbreak
- DAN mode
- forget rules

Todos os ataques foram bloqueados pelos guardrails implementados.

14. Execução do Sistema

Durante os testes, o sistema conseguiu: - bloquear ataques, - validar JSONs, - identificar urgência, - responder solicitações médicas.

15. Reflexão Final

O projeto demonstrou a importância da segurança em aplicações com LLMs. Foi possível compreender problemas relacionados a prompt injection, validação de dados e consistência de respostas. Também foi possível aprender sobre arquitetura modular, structured output e uso de modelos locais com Ollama. Em um ambiente real, seria importante adicionar autenticação, logs mais avançados e integração com APIs médicas.